| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |

**Aim:** To understand the process of convolution over the image and apply over the classification problem

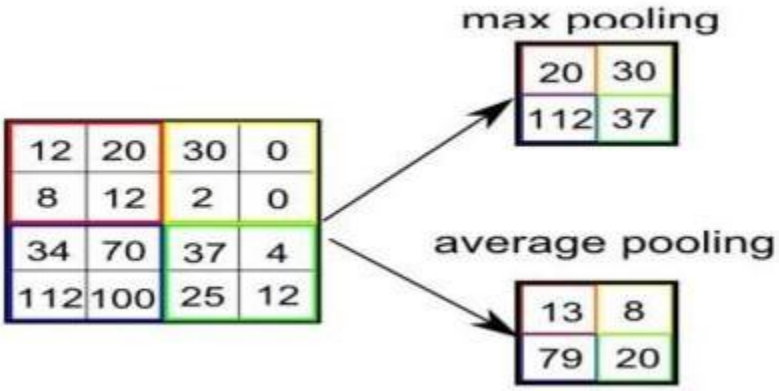
**IDE:** Google Colab

## Theory:

Convolutional Neural Networks (CNN) are complex feed forward neural networks. CNNs are used for image classification and recognition because of its high accuracy There are three types of layers in a convolutional neural network: i. Convolutional layer ii. Pooling layer iii. Fully connected layer Each of these layers has different parameters that can be optimized and performs a different task on the input data.

**What is Pooling Layer?**

Pooling layer is responsible for reducing the spatial size of the Convolved Feature. This is to decrease the computational power required to process the data through dimensionality reduction. There are two types of Pooling i. Average Pooling. ii. Max Pooling Max Pooling returns the maximum value from the portion of the image covered by the Kernel. On the other hand, Average Pooling returns the average of all the values from the portion of the image covered by the Kernel. Max Pooling also performs as a Noise Suppressant. It discards the noisy activations altogether and also performs de-noising along with dimensionality reduction. On the other hand, Average Pooling simply performs dimensionality reduction as a noise suppressing mechanism. Hence, we can say that Max Pooling performs a lot better than Average Pooling.




**What is Convolutional Layer?**

Convolutional layers are the major building blocks used in convolutional neural networks. A convolution is the simple application of a filter to an input that results in an activation. Repeated application of the same filter to an input results in a map of activations called a feature map, indicating the locations and strength of a detected feature in an input, such as an image. A convolutional layer contains a set of filters whose parameters need to be learned. The height and weight of the filters are smaller than those of the input volume. Each filter is convolved with the input volume to compute an activation map made of neurons.

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |

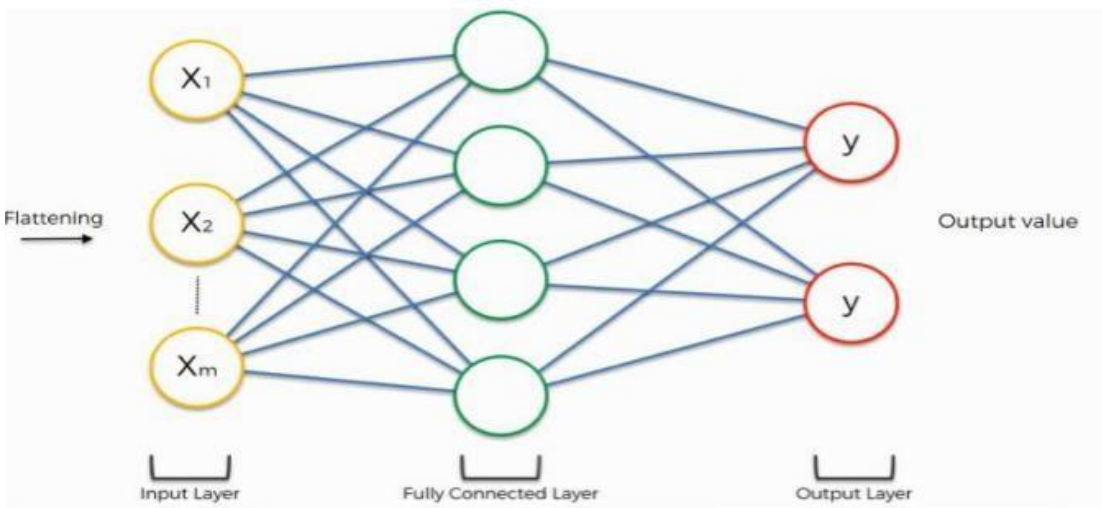
**What is Fully Connected Layer?**

A fully connected layer that takes the output of convolution/pooling and predicts the best label to describe the image We have three layers in the full connection step i. Input layer ii. Fully-connected layer iii. Output layer

**Input Layer**: It takes the output of the previous layers, "flattens" them and turns them into a single vector that can be an input for the next stage.

**Fully Connected Layer**: It takes the inputs from the feature analysis and applies weights to predict the correct label.

**Output Layer**: It gives the final probabilities for each label.




**ReLU Layer**: ReLU is an activation function. Rectified Linear Unit (ReLU) transform function only activates a node if the input is above a certain quantity, while the input is below zero, the output is zero, but when the input rises above a certain threshold, it has a linear relationship with the dependent variable. The main aim is to remove all the negative values from the convolution. All the positive values remain the same but all the negative values get changed to zero.

## Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Normalize the data
5. Pre-process the data
6. Visualize the Data

7. Write the CNN model function
8. Write the Cost Function
9. Write the Gradient Descent optimization algorithm
10. Apply the training over the dataset to minimize the loss
11. Observe the cost function vs iterations learning curve

## Program (Code):

```
# Step 1: Import libraries
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Step 2: Load dataset (Training dataset display - Requirement a)
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Show sample images
plt.figure(figsize=(6,6))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X_train[i], cmap='gray')
    plt.title(y_train[i])
    plt.axis('off')
plt.suptitle("Training Dataset Samples")
plt.show()

# Step 3: Normalize & reshape
X_train = X_train / 255.0
X_test = X_test / 255.0

X_train = X_train.reshape(-1,28,28,1)
X_test = X_test.reshape(-1,28,28,1)

y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)

# ------------------------------
# MODEL WITHOUT REGULARIZATION
# ------------------------------
```

```
model1 = models.Sequential([
   layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
   layers.MaxPooling2D((2,2)),
   layers.Flatten(),
   layers.Dense(128, activation='relu'),
   layers.Dense(10, activation='softmax')
])

model1.compile(optimizer='adam',
           loss='categorical_crossentropy',
           metrics=['accuracy'])

# Requirement b: Model summary
print("\nMODEL WITHOUT REGULARIZATION:")
model1.summary()

# Train
history1 = model1.fit(X_train, y_train_cat,
                 epochs=5,
                 validation_data=(X_test, y_test_cat))

# Requirement c & d: Accuracy & Loss before regularization
plt.plot(history1.history['accuracy'], label='Train Accuracy')
plt.plot(history1.history['val_accuracy'], label='Val Accuracy')
plt.title("Accuracy (Before Regularization)")
plt.legend()
plt.show()

plt.plot(history1.history['loss'], label='Train Loss')
plt.plot(history1.history['val_loss'], label='Val Loss')
plt.title("Loss (Before Regularization)")
plt.legend()
plt.show()

# ------------------------------
# MODEL WITH REGULARIZATION (Dropout)
# ------------------------------
model2 = models.Sequential([
   layers.Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)),
   layers.MaxPooling2D((2,2)),
   layers.Flatten(),
   layers.Dense(128, activation='relu'),
   layers.Dropout(0.5),
```

```
    layers.Dense(10, activation='softmax')
])

model2.compile(optimizer='adam',
               loss='categorical_crossentropy',
               metrics=['accuracy'])

print("\nMODEL WITH REGULARIZATION:")
model2.summary()

# Train
history2 = model2.fit(X_train, y_train_cat,
                      epochs=5,
                      validation_data=(X_test, y_test_cat))

# Requirement e & f: Accuracy & Loss after regularization
plt.plot(history2.history['accuracy'], label='Train Accuracy')
plt.plot(history2.history['val_accuracy'], label='Val Accuracy')
plt.title("Accuracy (After Regularization)")
plt.legend()
plt.show()

plt.plot(history2.history['loss'], label='Train Loss')
plt.plot(history2.history['val_loss'], label='Val Loss')
plt.title("Loss (After Regularization)")
plt.legend()
plt.show()

# ------------------------------
# Predictions
# ------------------------------
y_pred = model2.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

# Requirement g: Correct predictions
correct = np.where(y_pred_classes == y_test)[0]

plt.figure(figsize=(6,6))
for i, idx in enumerate(correct[:9]):
    plt.subplot(3,3,i+1)
    plt.imshow(X_test[idx].reshape(28,28), cmap='gray')
    plt.title(f"True:{y_test[idx]} Pred:{y_pred_classes[idx]}")
    plt.axis('off')
```

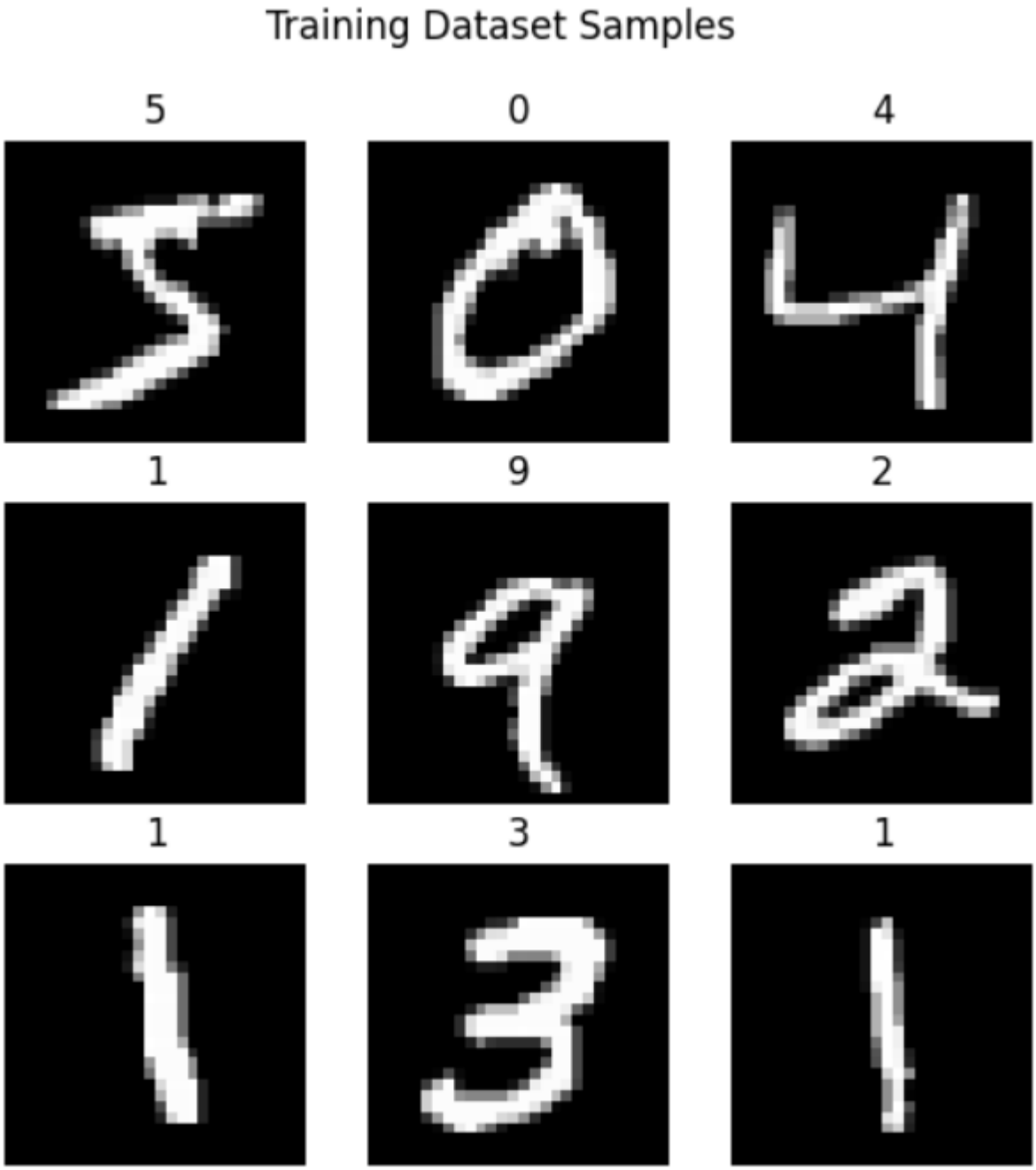
```
plt.suptitle("Correct Predictions")
plt.show()

# Requirement h: Incorrect predictions
incorrect = np.where(y_pred_classes != y_test)[0]

plt.figure(figsize=(6,6))
for i, idx in enumerate(incorrect[:9]):
    plt.subplot(3,3,i+1)
    plt.imshow(X_test[idx].reshape(28,28), cmap='gray')
    plt.title(f"True:{y_test[idx]} Pred:{y_pred_classes[idx]}")
    plt.axis('off')
plt.suptitle("Incorrect Predictions")
plt.show()
```

## Results:

a. Training dataset




b. Model summary

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |

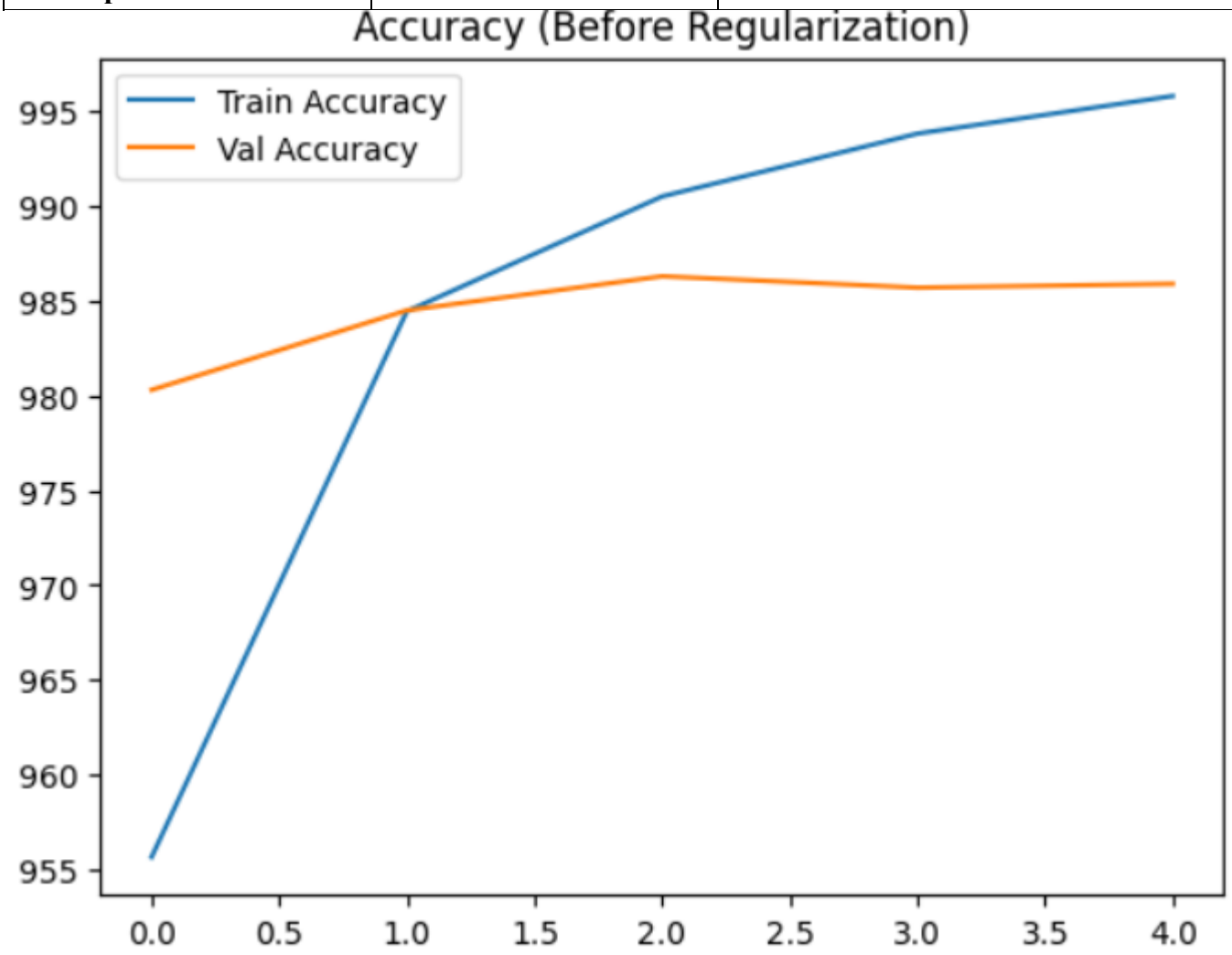
Model: "sequential"

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| flatten (Flatten) | (None, 5408) | 0 |
| dense (Dense) | (None, 128) | 692,352 |
| dense_1 (Dense) | (None, 10) | 1,290 |

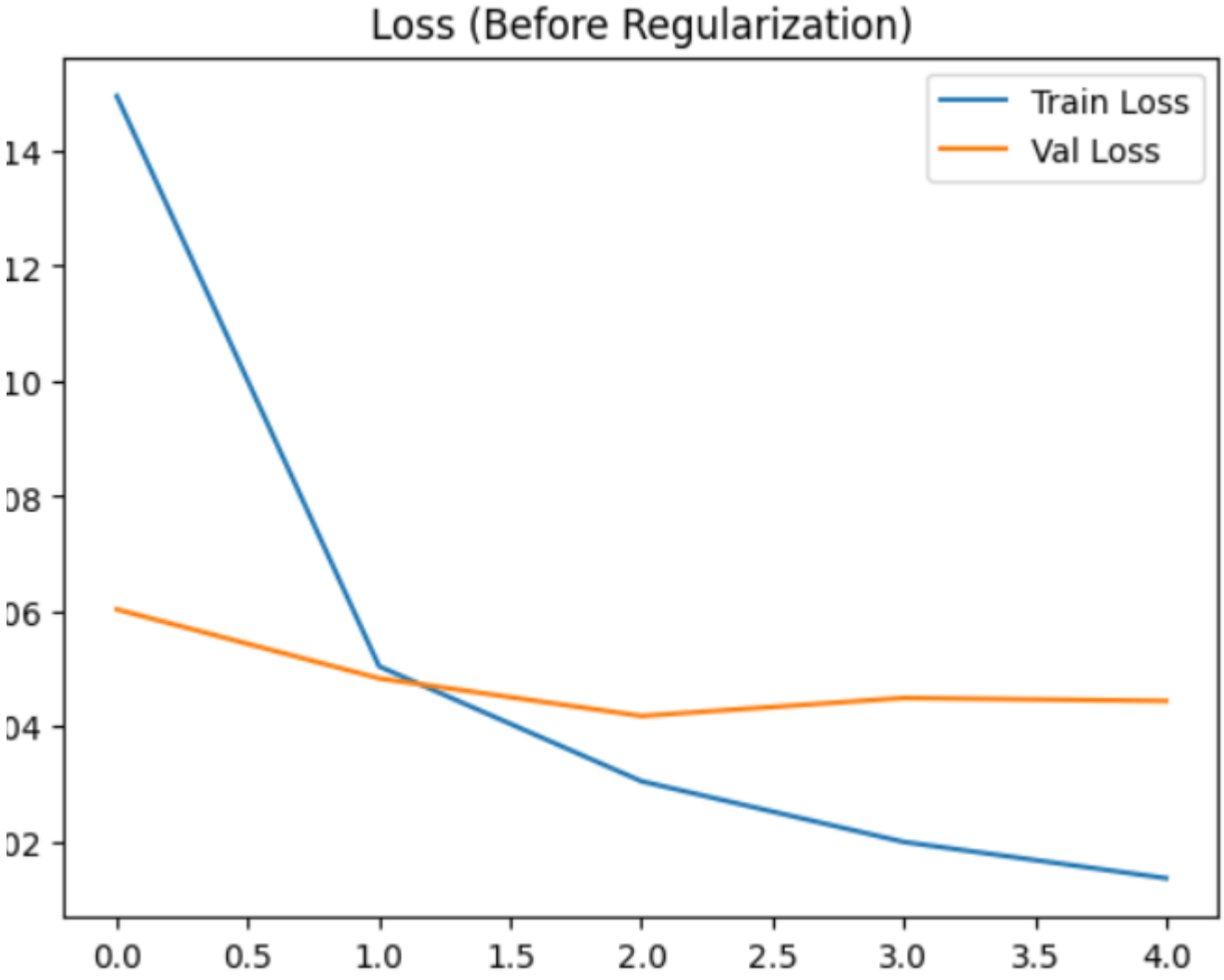
**Total params:** 693,962 (2.65 MB)
**Trainable params:** 693,962 (2.65 MB)
**Non-trainable params:** 0 (0.00 B)

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 47s 24ms/step - accuracy: 0.9556 - loss: 0.1495 - val_accuracy: 0.9803 - val_loss: 0.0603
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 43s 23ms/step - accuracy: 0.9844 - loss: 0.0504 - val_accuracy: 0.9845 - val_loss: 0.0483
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 43s 23ms/step - accuracy: 0.9905 - loss: 0.0305 - val_accuracy: 0.9863 - val_loss: 0.0418
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 46s 24ms/step - accuracy: 0.9938 - loss: 0.0199 - val_accuracy: 0.9857 - val_loss: 0.0449
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 79s 23ms/step - accuracy: 0.9958 - loss: 0.0136 - val_accuracy: 0.9859 - val_loss: 0.0444
```

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |



| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |



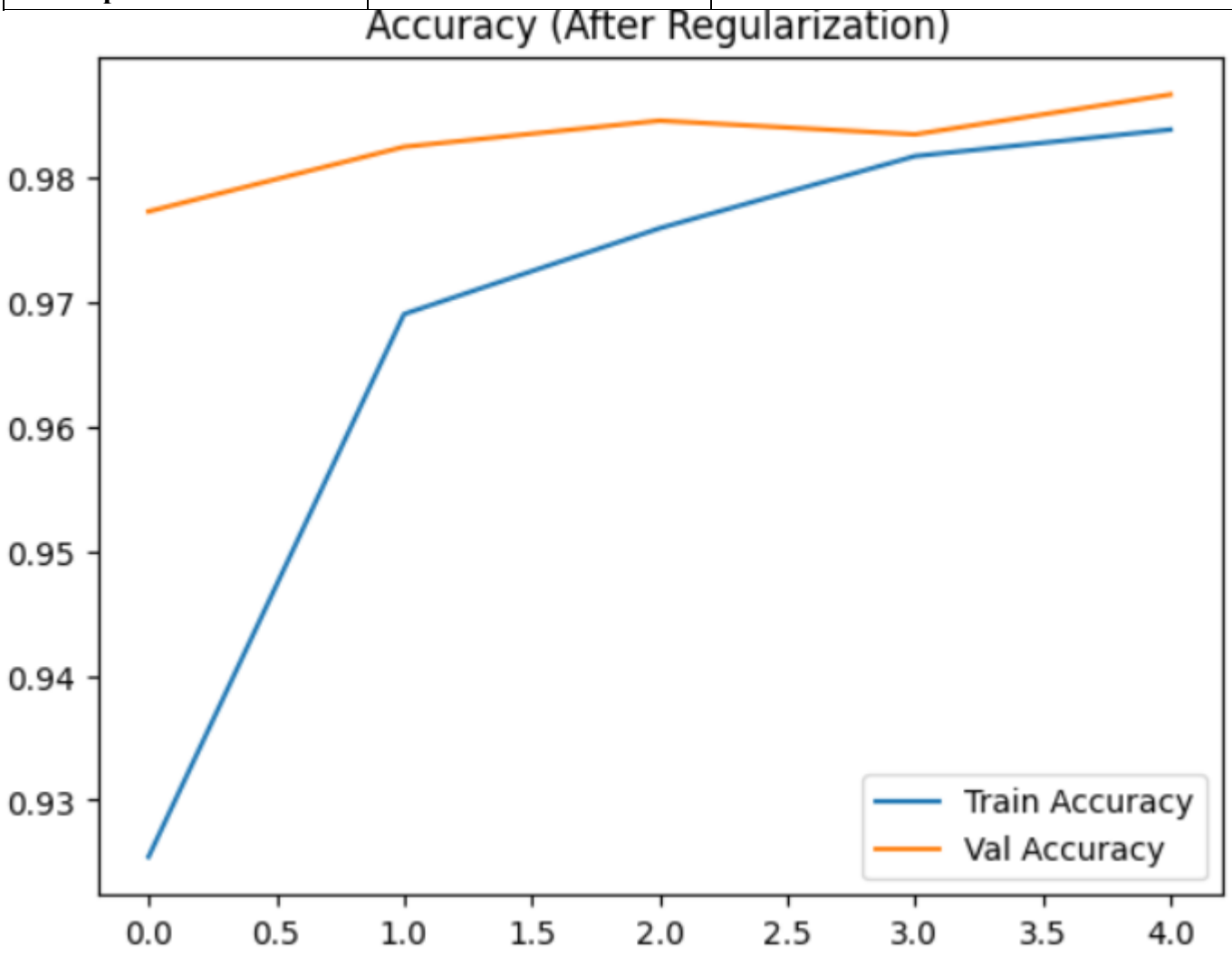
| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |

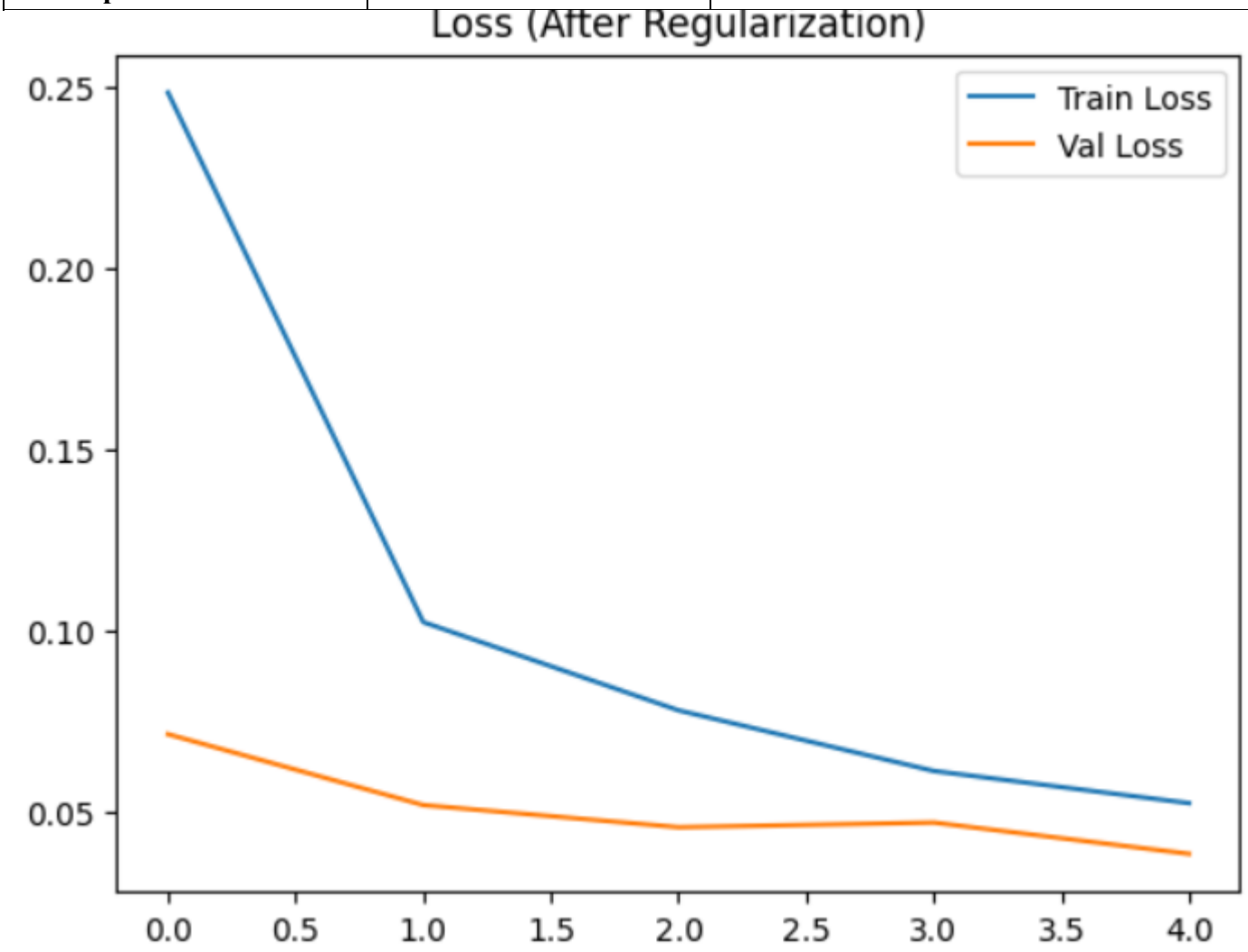
Model: "sequential_1"

| Layer (type) | Output Shape | Param # |
|---|---|---|
| conv2d_1 (Conv2D) | (None, 26, 26, 32) | 320 |
| max_pooling2d_1 (MaxPooling2D) | (None, 13, 13, 32) | 0 |
| flatten_1 (Flatten) | (None, 5408) | 0 |
| dense_2 (Dense) | (None, 128) | 692,352 |
| dropout (Dropout) | (None, 128) | 0 |
| dense_3 (Dense) | (None, 10) | 1,290 |

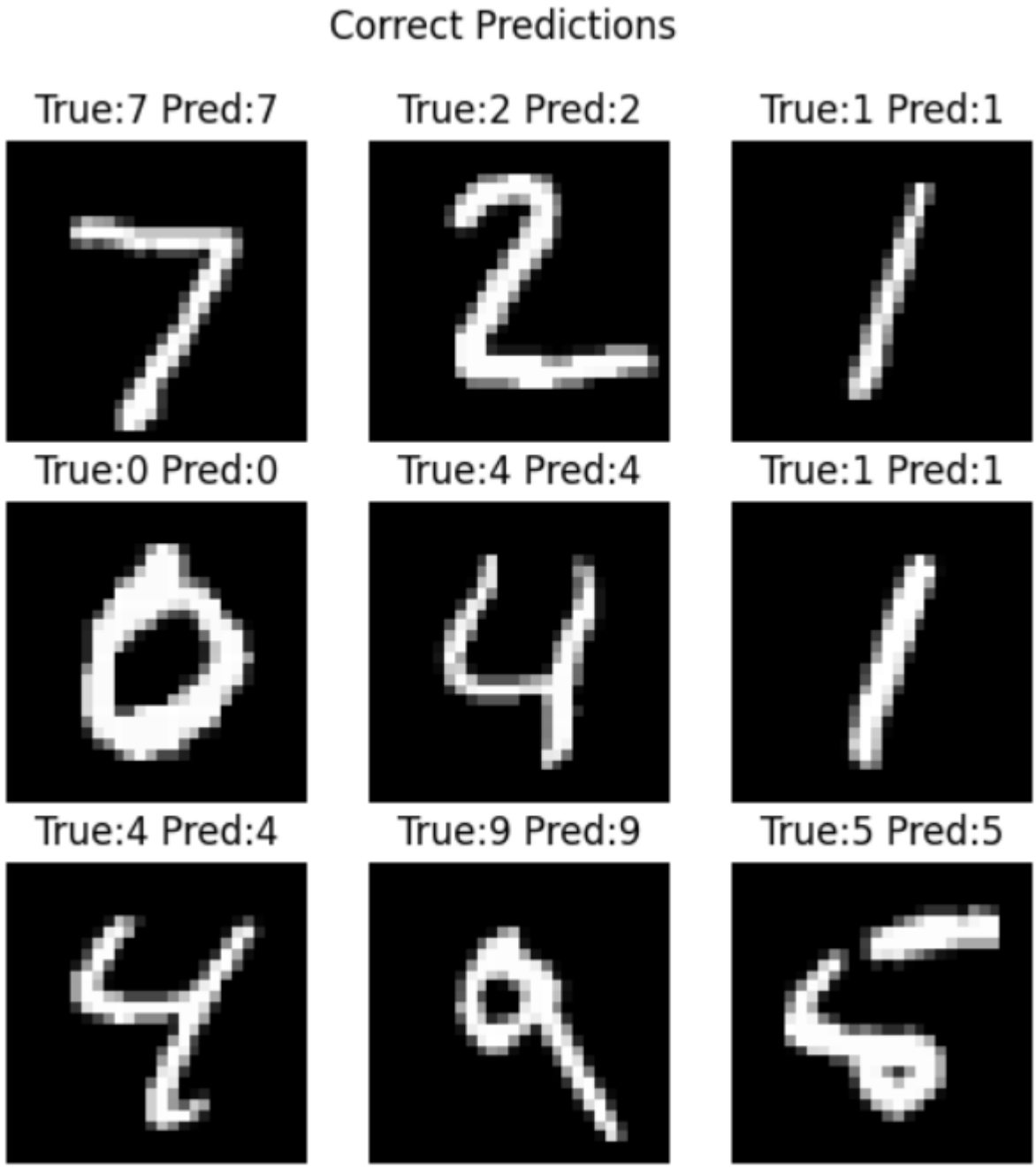
**Total params:** 693,962 (2.65 MB)
**Trainable params:** 693,962 (2.65 MB)
**Non-trainable params:** 0 (0.00 B)

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 45s 23ms/step - accuracy: 0.9254 - loss: 0.2484 - val_accuracy: 0.9773 - val_loss: 0.0714
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 23ms/step - accuracy: 0.9690 - loss: 0.1023 - val_accuracy: 0.9825 - val_loss: 0.0518
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 23ms/step - accuracy: 0.9760 - loss: 0.0780 - val_accuracy: 0.9846 - val_loss: 0.0457
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 23ms/step - accuracy: 0.9818 - loss: 0.0612 - val_accuracy: 0.9835 - val_loss: 0.0470
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 82s 23ms/step - accuracy: 0.9839 - loss: 0.0524 - val_accuracy: 0.9867 - val_loss: 0.0384
```

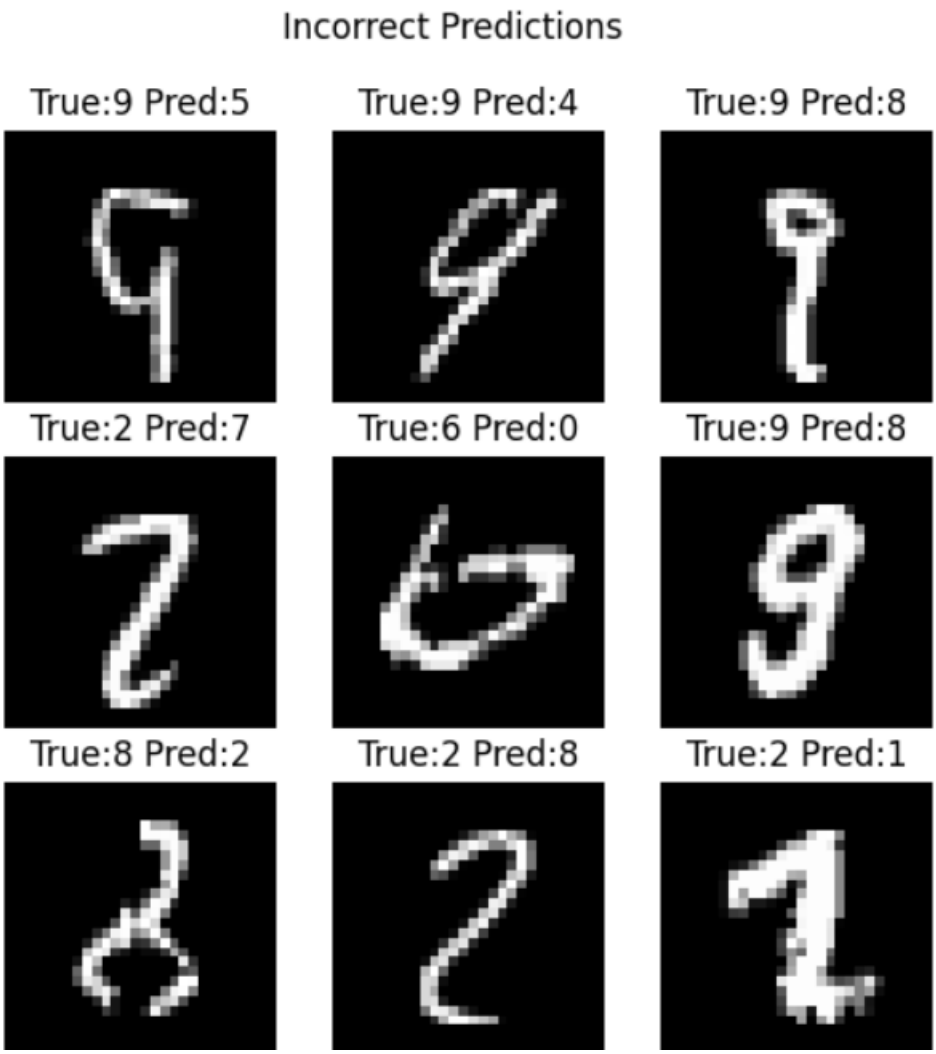

Accuracy (After Regularization)
0.98
0.97
0.96
0.95
0.94
0.93
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
Train Accuracy
Val Accuracy


Loss (After Regularization)
Train Loss
Val Loss
0.25
0.20
0.15
0.10
0.05
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |



| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |




## Pre Lab Exercise:

a. **What is Convolution process? Explain giving example.**

Convolution is the process of applying a filter (kernel) over an input image to extract important features such as edges, textures, or patterns.

It works by sliding the filter across the image and computing element-wise multiplication followed by summation.

Example:

If a 3×3 filter is applied to detect edges, it highlights boundaries in the image by emphasizing pixel intensity differences.

b. **What are different layers in CNN model?**

The main layers in a CNN are:

1. Convolutional Layer – Extracts features from input images
2. Pooling Layer – Reduces dimensions and computation
3. Fully Connected Layer – Performs classification

4. Activation Layer (ReLU) – Introduces non-linearity

c. **What is the requirement of the pooling layer in the CNN model?**

Pooling layer is used to:

- Reduce the size of feature maps
- Decrease computational cost
- Remove noise and redundant information
- Prevent overfitting

Example: Max pooling selects the maximum value from a region.

d. **What is the requirement of the use of ReLU activation function after convolution step?**

**ReLU (Rectified Linear Unit):**

- Converts negative values to zero
- Keeps positive values unchanged
- Introduces non-linearity
- Helps the model learn complex patterns faster

## Observation and Result Analysis:

e. **Nature of the dataset**

The dataset consists of grayscale images (28×28 pixels) representing handwritten digits (0–9). It is a labeled dataset used for image classification tasks.

f. **Training Process without regularization**

- The model learns patterns from the dataset
- Training accuracy increases rapidly
- Validation accuracy may fluctuate
- Slight overfitting may occur

g. **After regularization in the training Process**

- Dropout reduces overfitting
- Validation accuracy improves
- Model generalizes better
- Training becomes more stable

h. **Observation over the Learning Curves**

- Accuracy increases with epochs
- Loss decreases over time
- Gap between training and validation reduces after regularization
- Indicates improved model performance

| | | |
|---|---|---|
| Marwadi University | **Marwadi University**<br>**Faculty of Technology**<br>**Department of Information and Communication Technology** | |
| **Subject: Artificial Intelligence (01CT0616)** | **Aim: To understand the process of convolution over the image and apply over the classification problem** | |
| **Experiment No: 4** | **Date:02.04.2026** | **Enrolment No:92301733004** |

## Post Lab Exercise:

a. **Why CNN is preferred over ANN for images**

CNN is preferred because:

- It captures spatial features (edges, shapes)
- Uses fewer parameters than ANN
- Automatically extracts features
- More efficient for image data

b. **Can CNN be applied over Text data? If yes, then how. If no, then why?**

Yes, CNN can be applied to text data by:

- Converting text into numerical form (word embeddings)
- Applying convolution filters over sequences
- Extracting patterns like phrases or keywords

c. **What is the role of dropout layer?**

Dropout:

- Randomly disables neurons during training
- Prevents overfitting
- Improves generalization
- Makes the model more robust

d. **What will happen if maxpooling is replaced with minpooling?**

- Important features may be lost
- Model performance decreases
- Noise may increase
- Accuracy becomes lower